

Minimising k -Planarity with Simulated Annealing

SA, sa-stress, and the Adaptive-Budget Orchestrator

SAkGD — Graph Drawing Contest solver

July 6, 2026

Abstract

This document explains the three layers of the SAkGD solver: (1) the core **Simulated Annealing** (SA) engine that moves one node at a time to reduce edge crossings; (2) **sa-stress**, which warm-starts SA from a force-directed layout and is our strongest method on sparse graphs; and (3) the **orchestrator**, an adaptive scheduler that spends a fixed wall-clock budget across many graphs to minimise the worst k . The solver is a from-scratch re-implementation of the GD 2025 winning approach (Bianchetti & Moalic, *Winning the GD Challenge for the 4th Time*) plus our own additions.

1 The problem

Given a graph, place every node at an *integer* coordinate on a fixed grid so that (i) no two nodes share a point and (ii) no node lies on the interior of an edge it does not belong to. Edges are drawn as straight line segments. The objective is to minimise

$$k = \max_{e \in E} (\text{number of edges crossing } e),$$

i.e. the maximum number of crossings on *any single* edge. Ties are broken by the *total* crossing count $C = \frac{1}{2} \sum_e \text{xc}[e]$. Lower k means a “more planar” drawing; a planar graph achieves $k = 0$.

Shared engine. All methods sit on one incremental core:

- **Incremental crossing count.** A node move is scored without a global recount — only edges incident to the moved node and their crossing partners are updated (`planMove` / `commitMove`). The per-edge crossing sets `xs[e]`, counts `xc[e]`, the histogram `cntPerK[k]` and k itself are all maintained incrementally.
- **Spatial grid index.** Edges are bucketed into the cells their bounding box spans; crossing tests only run against edges sharing a cell.
- **Validity guard.** An `occupied` map forbids coincident nodes and a fast vertex-edge overlap test forbids a node landing on an edge it is not part of.

2 Simulated Annealing (SA)

SA follows Algorithm 1 of the paper and runs in **two phases**, each with its own fitness function and cooling parameters.

2.1 Two phases

Phase 1 — minimise total crossings. Fitness is the change in total crossings ΔC . A worsening move ($\Delta C > 0$) is accepted with the Metropolis probability $\exp(-\Delta C/T)$. This phase does the coarse untangling.

Phase 2 — minimise the k -value. A dual-level fitness: the *primary* term is the change in the *local* maximum crossing count among the incident edges and their crossing partners; the *tie-breaker* is the normalised change in total crossings. This phase specifically attacks the bottleneck edges that set k .

2.2 Move proposal and the wave structure

Each step proposes moving one node:

- **Node selection** (`selectNode`) is biased toward nodes touching heavily-crossed edges, weight $\approx 1 + \sum xc[e]$.
- **Position selection** (`selectPlace`) samples a Gaussian around the current position with $\sigma \propto \sqrt{T/T_0}$, plus a 5% chance of a global random jump in Phase 1.
- **Waves.** The run is divided into *waves*. At the start of each wave the layout is restored to the best-known solution and the temperature is clipped by $decTW$. This lets the search cool, explore, then relaunch from the incumbent best — the sawtooth in Figure 1 (left).

The temperature decays geometrically by $decT$ every step and is reset per wave by $decTW$. As T falls, the acceptance probability of any worsening move collapses (Figure 1, right): early on the search wanders freely; late on it behaves like greedy hill-climbing.

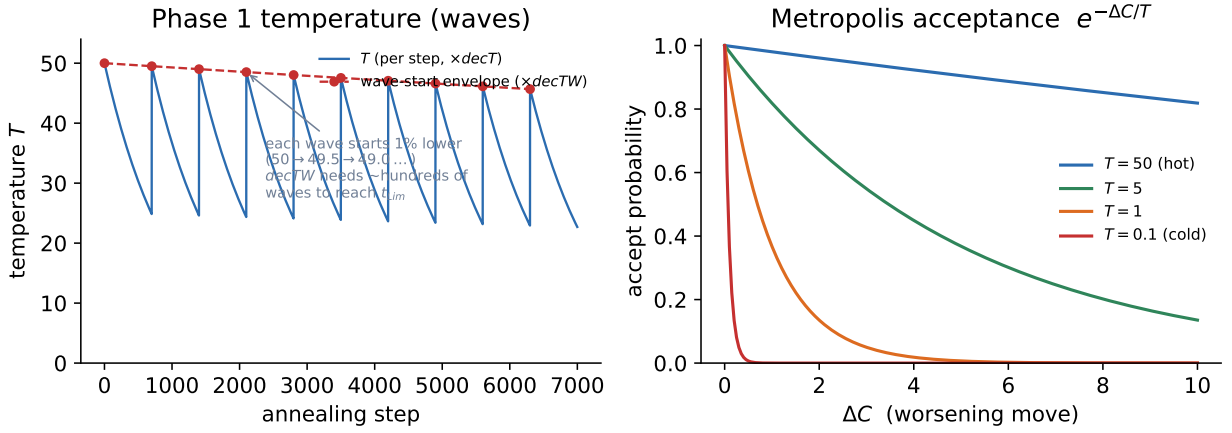


Figure 1: **Left:** the wave-based temperature schedule for Phase 1 ($T_0=50$, $decT=0.999$, $decTW=0.99$). Two decays act at once: *within* a wave the temperature free-falls as $T \times decT$ every step (blue); *between* waves only the wave-start temperature drops, by $decTW$ (red envelope). Because $decTW$ is just a 1% cut per wave, the peaks barely move — it takes hundreds of waves to reach t_{Lim} , which is what ends the phase. **Right:** the Metropolis acceptance curve $e^{-\Delta C/T}$ — hot temperatures accept large worsening moves, cold temperatures reject almost everything.

Table 1: SA cooling parameters (paper Table 1).

Phase	objective	T_0	$decT$	$decTW$	t_{Lim}
1	total crossings ΔC	50	0.999	0.99	0.01
2	k -value (local max)	1	0.9999	0.99	0.01

3 sa-stress: force-directed initialisation + SA

SA is only as good as where it starts. The single largest improvement over the baseline came from replacing the constructive BFS-snake initial layout with a **force-directed / stress-minimisation** layout. This is the paper’s step 1 (stress majorization + repeated FMMM, keep the best candidate) rebuilt *without* OGDF — whose Python bindings do not work on macOS — using the Graphviz CLI instead.

sa-stress runs two stages, warm-chained:

1. `tools/stress_init.py` ($\approx 8\%$ of budget). Runs Graphviz **sfdp** (multilevel force-directed, the FMM analogue) and, on small graphs, **neato** (stress majorization, the SM analogue), up to 8 attempts. Each attempt is randomly rotated, scaled to the grid, snapped to integers (collisions pushed to the nearest free cell by a spiral search), and scored by a *sampled* crossing density ($\sim 200k$ random edge pairs via NumPy). The best candidate is written. For $n > 3000$ only sfdp is used (faster and consistently better in our experiments).
2. **sakgd** (remaining $\approx 92\%$). The two-phase SA above, warm-started from the stress layout. If the init stage fails, the BFS-snake safety net takes over automatically.

When it helps — and when it hurts. On *sparse* graphs (A7/A8/A9, $m/n \approx 2\text{--}3.5$, near-planar densities) a good global embedding drops k by $3\text{--}4\times$ before SA even starts; SA only polishes. On the *dense* graph A6 ($m/n = 15$) stress init actually *hurts* — there is no low-crossing embedding to find, and the force layout starts worse than a plain random/BFS start (Figure 2). For dense graphs the orchestrator therefore falls back to plain **sa**.

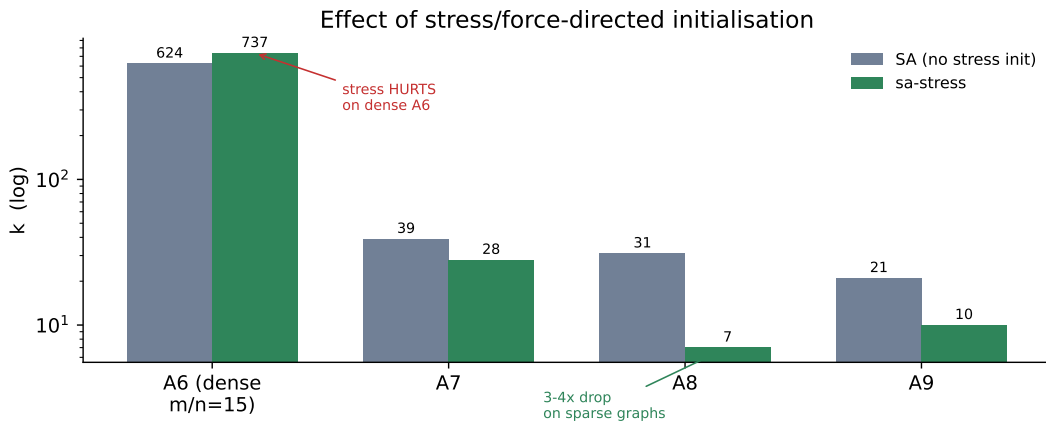


Figure 2: Stress/force-directed initialisation collapses k on sparse graphs but regresses on the dense A6 (note the log scale). This is why method choice is density-dependent.

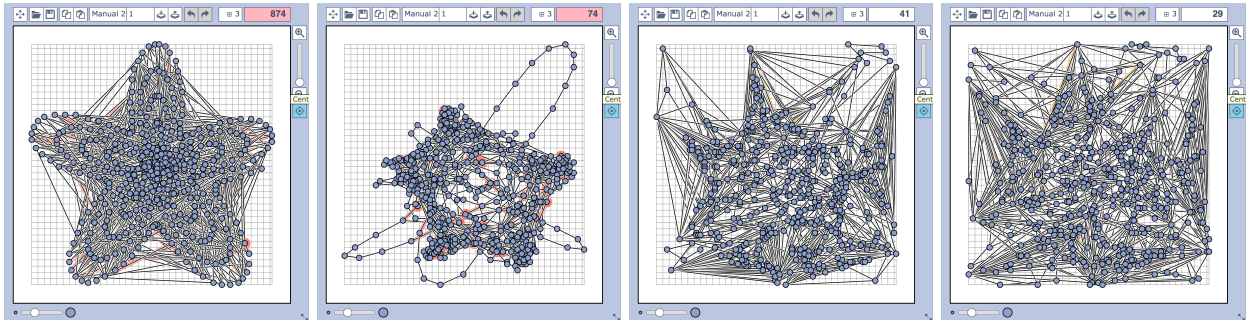


Figure 3: The **sa-stress** pipeline on Automatic-7, left to right: (1) raw input layout, (2) after stress/sfdp initialisation, (3) after SA Phase 1 (total-crossing reduction), (4) after SA Phase 2 (k -value reduction, the final drawing).

4 Other methods in the portfolio

Besides **sa** and **sa-stress**, the batch runner registers alternative metaheuristics (the paper uses SA only):

- **lns** — Large Neighbourhood Search: moves a BFS-connected *group* of K nodes per iteration, trying R candidate positions each and committing only strictly-improving moves. Escapes some

local optima but stalls at a local optimum (it never accepts a worsening move).

- **ils** — Iterated Local Search: alternates a full two-phase SA run with a random *kick* that relocates $\sim n/10$ nodes, keeping the global best. Directly addresses the LNS stall.
- **staged** — LNS for 30% of the budget (fast coarse descent) then SA warm-started on that output for the remaining 70%.

After stress-init was added these lost most of their edge, but they remain in the portfolio. Two literature-backed ideas (`--lexk`, `--krepair`) were tested, lost in ablation, and are kept behind default-off flags.

5 The adaptive-budget orchestrator

`orchestrate.py` answers a different question: given *one* wall-clock budget (e.g. 1 hour) and 8–10 graphs, how do we spend it to minimise the *worst* k across all of them? Spending equal time per graph is wasteful — some graphs converge in seconds while others (dense A6) keep improving for the whole hour. The orchestrator is a **marginal-gain scheduler**: it measures how fast each graph is still improving and pours time into whichever graph has the steepest slope.

5.1 How it works

Classify. From (n, m) pick a method per graph: dense/expander ($m/n \geq 8$) $\rightarrow$ **sa**; sparse/huge $\rightarrow$ **sa-stress**; Graphviz-missing $\rightarrow$ **sa**. These thresholds are *soft*, *CLI-overridable* priors that bias only the cold-lease init — never the scheduler itself.

Quantum leasing. Time is handed out in quanta $Q_b = \text{clamp}(B/(4n_{\text{live}}), 45, 180)$ s via the existing W -worker multi-start `run_combo`. Huge graphs get a longer floor and fewer workers on their first (cold) lease to relieve memory pressure.

Explore. Each graph gets one quantum, hardest-first, so every graph yields a real dk/dt slope and the huge graph pre-pays its expensive setup. Explore is capped at 40% of the budget B .

Convergence / early termination. The best-of-workers envelope is read from each graph’s `.trace`. A graph is dropped when its k -slope is 0 *and* total crossings fell $< 0.5\%$ over a ≥ 150 s window (with minimum-lease/tick guards). Hard-drop on $k=0$ or temperature floor. Convergence is decided purely from the measured trace — no graph-class exemptions.

Reallocation. Banked time goes to the top bidder, $\text{bid} = (dk/dt) \cdot Q$, via a *warm-chained* lease (**sa-warm**: a single `sakgd` stage with `--init input`) resuming the graph’s verified best layout. Anti-starvation serves the least-served among close bidders; a minimax sink targets the worst- k graph when nobody improves.

Budget safety. A deadline-aware per-process timeout guarantees no lease runs past budget; every graph gets a verified submission (falling back to its valid input if needed).

5.2 Why warm-chaining is cheap

A key empirical fact makes this work: resuming a *good* layout with `--init input` is cheap even on the 10k-node A8 — `computeAllCrossings` scales with the *current* crossing count, not the node count, so a warm continuation of a $k=6$ layout costs ~ 3 s of setup. Only the raw, tangled input is expensive. Hence every lease after the first is a cheap, monotone continuation (Figure 5). A killed `sakgd` writes no output, so the scheduler never kills a worker — the per-lease timeout is only a budget backstop, and a lost quantum simply keeps the previous warm layout.

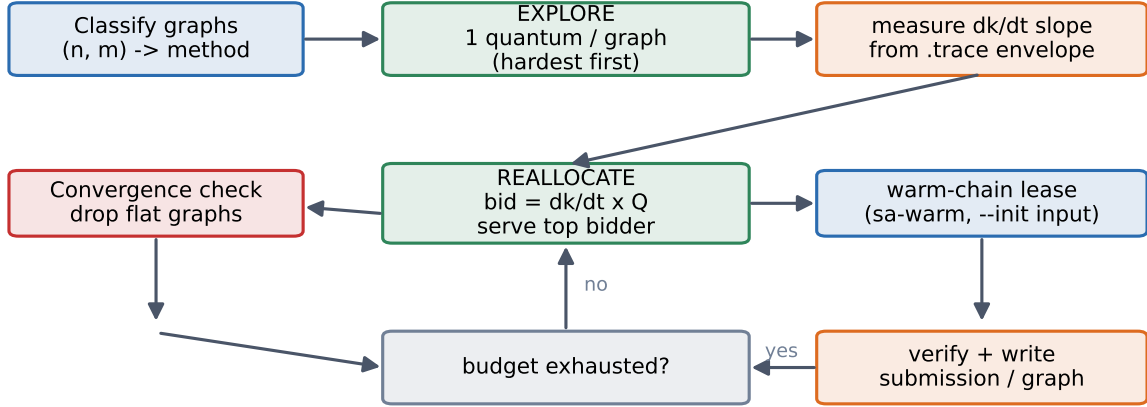


Figure 4: The orchestrator loop: explore once per graph to measure slopes, then repeatedly reallocate banked time to the steepest-improving graph via warm-chained leases, dropping graphs that have gone flat, until the budget is exhausted.

6 Results

Table 2 and Figure 6 summarise the current best k per contest graph. On the sparse graphs the wins come almost entirely from stress initialisation (A8: $31 \rightarrow 7$, A9: $21 \rightarrow 10$, A7: $39 \rightarrow 28$), beating both the rival team and the GD’25 winner. On the dense A6, where stress init does not help, the only effective lever is budget, and the orchestrator’s job is to direct that budget there.

Table 2: Best k per graph. Lower is better; **bold** = our current best.

Graph	n / m	our old best	our new best (method)	rival	GD’25 winner
Automatic-6	200 / 3000	632	624 (sa, 15 min)	621	568
Automatic-7	500 / 1740	39	28 (sa-stress)	31	31
Automatic-8	10466 / 20288	31	7 (sa-stress)	12	15
Automatic-9	2519 / 4938	21	10 (sa-stress)	13	12

Quick usage

- Single sparse graph: `python3 tools/stress.init.py -i g.json -o init.json; ./sakgd -i init.json -o out`
- Full portfolio: `python3 run_contest.py --methods sa-stress,sa --graphs 6-9 --workers 8`
- Adaptive 1-hour run: `python3 orchestrate.py --graphs 1-9 --budget 3600 --workers 8`

Reference: Bianchetti, J. & Moalic, L. (2025). *Winning the GD Challenge for the 4th Time: Our Approach*. GD 2025, LIPIcs.GD.2025.43. See also `docs/YAKLASIMLAR.md` and `results/exp-2026-06-16-orchestrator.md`.

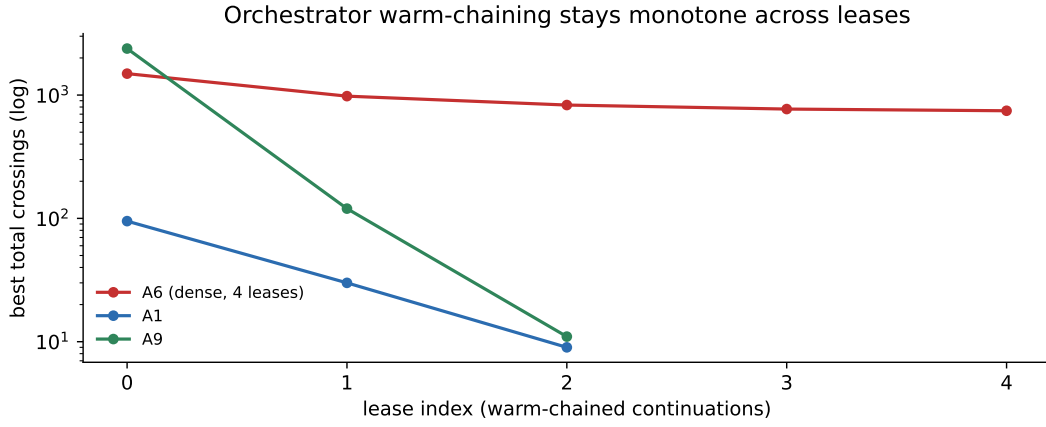


Figure 5: Warm-chained leases stay monotone (data from a $B=220$ s end-to-end run). The dense A6 measured as still-improving and won the most budget (4 leases); the sparse graphs converged fast and were deprioritised.

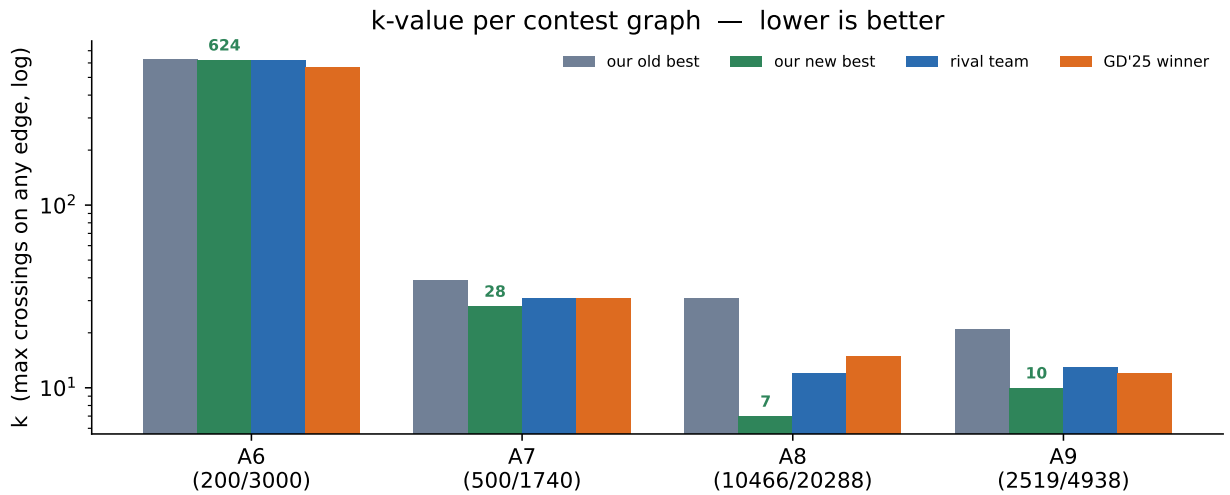


Figure 6: k -value per graph across four solvers (log scale). Our new bests (green) beat the rival and the GD'25 winner on all three sparse graphs; A6 remains the open frontier.